

OOP Basics

What is OOP?

- Object Oriented Programming (OOP) is a programming approach
- Uses "objects" to design applications and computer programs
- It utilizes classes and objects to model real-world entities, making code more modular, reusable, and easier to maintain

What is an Object?

- An object is a bundle of state and behavior
- State - Attributes define the state of an object
- Behavior - Methods define the behavior of an object

What is a Class?

- A class is a **blueprint** for creating objects
- Every object has a class
- Contains attributes (state) and methods (behavior)
- Example -

```
1. public class Car {  
2.     public int speed;    //attribute  
3.     public String color; //attribute  
4.     //method (behavior)  
5.     public void accelerate() {  
6.         System.out.println("This is a method");  
7.     }  
8. }
```

State of a Class

- Attributes define the state of an object
- Attributes are defined as variables within the class
- Example: In the `Car` class, color and speed are attributes
- The color and speed attribute define the state of a car object

- 1. `public class Car {`
 2. `public int speed; //attribute`
 3. `public String color; //attribute`
 4. `}`

Behavior of a Class

- Methods define the behavior of an object.
- Example: In the Car class, accelerate is a method.
- Methods perform operations on the attributes or provide functionality.
- Example:

```
1. public class Car {  
2.     public int speed;    //attribute  
3.     public String color; //attribute  
4.     //method (behavior)  
5.     public void accelerate() {  
6.         speed += 10;  
7.     }  
8. }
```

Lets create an Object

- To create an object we write

```
1. Car toyota = new Car();  
2.  
3. // ClassName variableName = new ClassName();
```

- This creates an instance of our `Car` object
- Created using the 'new' keyword
- Implementing the Class design we can create many instances of it
- Recall Scanner and check if you can find any similarities

Why we need a Tester Class?

- Java code starts executing from the main method
- We will run the main method from a Tester class
- For now we will keep our design Classes and Tester class separate
- We can have many classes and create their objects in the Tester class

Accessing Attributes/Fields

- Now let's define the state of the `Car` object named `toyota`
- Use the dot (.) operator to access or set fields/attributes

```
1. public class Car {  
2.     public int speed;    //attribute  
3.     public String color; //attribute  
4. }
```

```
1. public class Tester {  
2.     public static void main(String [] args){  
3.         Car toyota = new Car();  
4.         //Setting attribute  
5.         toyota.color = "white";  
6.         toyota.speed = 20;  
7.         //Accessing attribute  
8.         System.out.println(toyota.speed); // 20  
9.     }  
10. }
```

Objects and Memory Location

- When an object is created, it is stored in the heap memory.
- The variable that holds the object is actually a reference (or pointer) to the memory location of the object.
- Each new object created has a different memory location.
- Multiple variables can reference the same object.
- Changing the object through one reference affects all references to that object.

Objects and Memory Location Example

```
1. public class Person {  
2.     public String name; //attribute  
3. }
```

```
1. public class Tester {  
2.     public static void main(String [] args){  
3.         Person p1 = new Person();  
4.         p1.name = "Bob";  
5.         Person p2 = new Person();  
6.         p2.name = "Bob";  
7.         System.out.println(p1);  
8.         System.out.println(p2);  
9.     }  
10. }
```

```
Welcome to DrJava. Working directory is /CSE111  
> run Tester
```

```
Person@1b28cdfa  
Person@eed1f14
```

- Printing the object shows us the location of where the object is stored.
- With the new keyword, a new object is created and allocated a new address

Objects and Memory Location Example

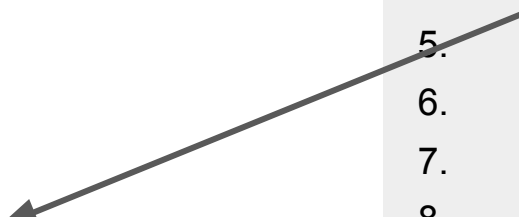
```
1. public class Person {  
2.     public String name; //attribute  
3. }
```

```
1. public class Tester {  
2.     public static void main(String [] args){  
3.         Person p1 = new Person();  
4.         p1.name = "Bob";  
5.         Person p2 = new Person();  
6.         p2.name = "Bob";  
7.         System.out.println(p1.name == p2.name); // true  
8.         System.out.println(p1 == p2); // false  
9.         Person p3 = p1;  
10.        System.out.println(p1 == p3); // true  
11.    }  
12. }
```

Objects and Memory Location Example (Contd.)

```
1. public class Person {  
2.     public String name; //attribute  
3. }
```

```
1. public class Tester {  
2.     public static void main(String [] args){  
3.         Person p1 = new Person();  
4.         p1.name = "Bob";  
5.         Person p2 = new Person();  
6.         p2.name = "Bob";  
7.         System.out.println(p1.name == p2.name); // true  
8.         System.out.println(p1 == p2); // false  
9.         Person p3 = p1;  
10.        System.out.println(p1 == p3); // true  
11.    }  
12. }
```



Both of the Person objects have different memory location, despite having the same value for their name field. They do not point to the same reference!

Objects and Memory Location Example (Contd.)

```
1. public class Person {  
2.     public String name; //attribute  
3. }
```

Person P3 now points to the same reference as P1,
They are now referencing the same object.

```
1. public class Tester {  
2.     public static void main(String [] args){  
3.         Person p1 = new Person();  
4.         p1.name = "Bob";  
5.         Person p2 = new Person();  
6.         p2.name = "Bob";  
7.         System.out.println(p1.name == p2.name); // true  
8.         System.out.println(p1 == p2); // false  
9.         Person p3 = p1;  
10.        System.out.println(p1 == p3); // true  
11.    }  
12. }
```

Try for yourself

- Update the name for p3 to “joe”
- Print the name for p1 and p3
- Look at the output. Did anything change for the object p1?

```
1. p3.name = “joe”;  
2. System.out.println(p3.name); //???  
3. System.out.println(p1.name); //???
```

THANK YOU!